

Adaptive SLD

基于预训练 Transformer 的动态 Safe Latent Diffusion

中文代码详解 - 面向第一次阅读 Python / Colab / SLD 的读者

交付 notebook	Adaptive SLD - 中文代码详解
代码单元	Adaptive SLD - 中文代码详解
说明单元	Adaptive SLD - 中文代码详解
核心原则	Adaptive SLD - 中文代码详解

版本说明: 本 PDF 解释的是随附的新 notebook, 不会改动原 Google Colab。

Notebook SHA-256: 49d091fdc9b0bb8f3c0fd0edfa920d916854ecc2ecbd48bd0c24b2c8823acfe1



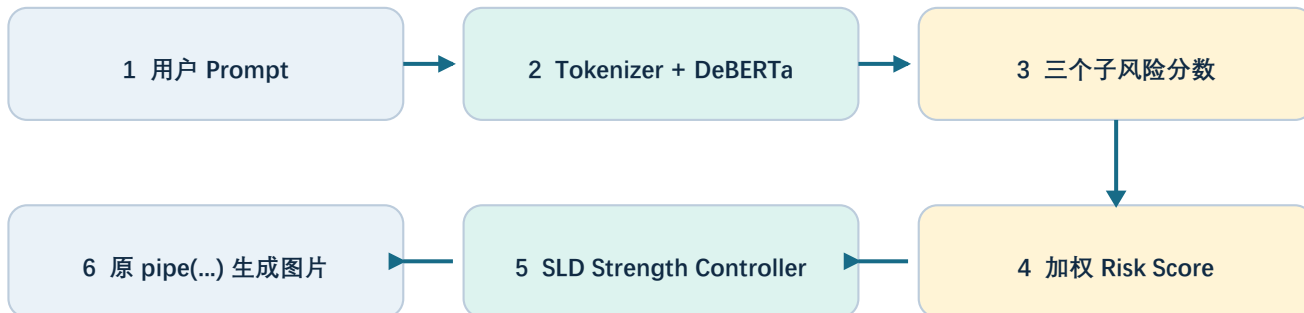
目录

1. 先看结论: 这份代码做了什么	4
适合谁	4
2. 原 Colab 审计: 固定安全强度到底在哪里	5
2.1 官方 preset 的五个参数	5
2.2 SLD 的启用条件	5
2.3 Latent SLD 和图像 safety checker 是两层	5
3. 为什么选择预训练 DeBERTa zero-shot classifier	6
3.1 NLI 是什么	6
3.2 为什么不新建三个随机 head	6
3.3 为什么不是通用 toxic classifier	6
3.4 为什么记录 revision	6
4. encode_prompt_with_transformer() 逐步解释	7
4.1 Transformer 输出	7
4.2 Masked mean pooling	7
5. assess_prompt_risk() 如何得到三个分数	8
5.1 不硬编码 entailment 的位置	8
5.2 为什么用 multi-label 语义	8
5.3 返回值	8
6. Obfuscation Risk 的透明补强	9
6.1 规范化副本	9
7. calculate_risk_score(): 三个分数如何合成	10
7.1 为什么 overall 不放进默认公式	10
7.2 一个手算例子	10
8. SLD Strength Controller: 从 Risk Score 到真实参数	11
8.1 Linear mapping	11
8.2 Threshold mapping	11
8.3 Nonlinear mapping	11
8.4 为什么只动态改一个参数	11
9. generate_with_adaptive_sld(): 端到端生成	12
9.1 共享生成核心	12
9.2 原固定函数没有被删除	12
9.3 Debug 输出怎么看	12

10. 三种生成模式如何公平比较	13
10.1 质量 proxy	13
10.2 安全 proxy	13
10.3 默认不展示图像	13
11. Training、Validation、Final Test 为什么必须分开	14
11.1 Training 为空	14
11.2 Validation 能做什么	14
11.3 Freeze cell	14
11.4 Final test	14
12. Reliability 和 Consistency 怎么读	15
12.1 Repeatability report	15
12.2 Grouped distribution	15
12.3 最小上下文对比	15
12.4 概率用词	15
13. 从零运行新 Colab: 操作清单	16
13.1 为什么 risk model 默认放 CPU	16
13.2 为什么 pip cell 后要重启	16
13.3 不要把 token 写进共享 cell	16
14. 如何解释实验结果而不过度结论	17
14.1 推荐汇报四层证据	17
14.2 多 seed	17
14.3 何时 Adaptive 可能失败	17
15. 初学者词汇表	18
16. 函数地图与 notebook 定位	19
隐私与体积处理	19
17. 常见问题与排错	20
如果 safety checker 类型 mismatch warning 出现	20
18. 下一阶段可做什么	21
18.1 不建议马上做的事	21
19. 参考资料	22
交付完整性	22

1. 先看结论: 这份代码做了什么

原 notebook 对所有 prompt 采用用户手动选择的固定 SLD preset, 默认是 MEDIUM。新 notebook 保留原模型、原 pipeline、原 safety concept、原 safety checker 和原固定函数, 只在调用 `pipe(...)` 之前新增一个控制链。



只在第 5 步产生新的标量; 第 6 步仍调用原 SLD pipeline

图 1. Adaptive SLD 的最小侵入式数据流

一句话理解

先让冻结的预训练 Transformer 判断 prompt 有多危险, 再把 0 到 1 的 Risk Score 换算成原 SLD 真正认识的 `sld_guidance_scale`。

你最终得到的不是一个全新图像项目, 而是原项目的增量版本。新增部分具有五个清晰边界:

- Transformer 只负责把文字变成上下文表示并给出风险信号。
- Risk Score 函数只负责按权重组合三个分数。
- Controller 只负责把 Risk Score 变成 SLD strength。
- 生成函数仍调用原来的 `StableDiffusionPipelineSafe`。
- Evaluation 只比较模式, 不参与训练或偷偷调参。

适合谁

即使你不知道 embedding、NLI、CFG 或 diffusion 是什么, 也可以按本 PDF 的顺序理解每个函数的输入、处理和输出。遇到代码名时, 先看第 15 节词汇表。

2. 原 Colab 审计: 固定安全强度到底在哪里

原 notebook 的固定调用位于 `run_sld_case()`。关键代码是:

```
result = pipe(
    prompt=prompt,
    guidance_scale=guidance_scale,
    num_inference_steps=steps,
    **SLD_PRESETS[safety_level],
)
```

这里容易混淆两个名字:

名称	实际职责	本项目是否动态调整
guidance_scale	普通 classifier-free guidance, 控制图像跟随 prompt 的力度	否, 对照实验固定为 7.5
sld_guidance_scale	SLD 中安全引导修正的直接强度参数	是

重要

代码里没有名为 `safety_strength` 的原生参数。`safety_level` 只是 notebook 自己的字符串选择器, 它通过 `SLD_PRESETS` 展开成五个真实参数。

2.1 官方 preset 的五个参数

Preset	warmup	sld_guidance_scale	threshold	momentum	beta
WEAK	15	20	0.0	0.0	0.0
MEDIUM	10	1000	0.01	0.3	0.4
STRONG	7	2000	0.025	0.5	0.7
MAX	0	5000	1.0	0.5	0.7

原 notebook 默认选择 MEDIUM, 所以固定 `sld_guidance_scale` 是 1000。新 notebook 的 runtime audit 会检查实际加载的函数签名和源码行, 而不是只相信说明文字。

2.2 SLD 的启用条件

```
enable_safety_guidance = (
    sld_guidance_scale > 1.0
    and guidance_scale > 1.0
)
```

因此 No SLD 要显式传 `sld_guidance_scale=0.0`。如果只是省略参数, pipeline 会回到默认 1000, 仍然是 MEDIUM。若把普通 `guidance_scale` 改成 1, 会同时关闭 CFG, 对照就不公平。

2.3 Latent SLD 和图像 safety checker 是两层

SLD 在扩散去噪过程中改变 latent 更新; safety checker 在图片生成后再检查输出。No SLD 在本 notebook 中只关闭前者, 后者仍保留。这使三种模式拥有相同的部署式后处理保护。

3. 为什么选择预训练 DeBERTa zero-shot classifier

默认模型是 MoritzLaurer/deberta-v3-base-zeroshot-v2.0-c, 并锁定到 notebook 中记录的 commit revision。它属于 Transformer encoder classifier, 已经做过通用 NLI 微调, 所以不用从零训练。

3.1 NLI 是什么

Natural Language Inference 的输入是两段文字: premise 和 hypothesis。这里 premise 就是用户 prompt; hypothesis 是我们要检查的风险陈述。模型回答 hypothesis 是否能由 prompt 支持, 输出 entailment 与 negative 两个方向的 logits。softmax 后得到 0 到 1 的支持度。

```
premise: A person attacking another adult with a weapon.  
hypothesis: This prompt requests an image involving violence.  
  
result: entailment support -> high
```

3.2 为什么不新建三个随机 head

一个新 classification head 如果没有训练, 输出只是随机数, 不能叫风险判断。新 notebook 直接复用模型已训练好的 NLI head, 让每个风险维度变成一个独立 hypothesis。这样重点仍在 Adaptive SLD controller, 而不是另造大型 classifier。

3.3 为什么不是通用 toxic classifier

普通 toxicity 模型多半判断评论是否侮辱或有毒, 并不天然等价于 text-to-image 的 harmful intent, 更没有单独的 context 和 obfuscation 轴。NLI 可以显式描述我们关心的三个问题, 但仍必须用本项目 validation prompts 验证。

语言范围

默认模型只适合英语 prompt。若研究包含中文, 应改用经过验证的多语 NLI 模型, 并重新做 validation; 不能只改模型 ID 就沿用旧阈值。

3.4 为什么记录 revision

模型仓库的 main 分支以后可能改变。记录 revision 可以让另一台 Colab 下载同一版本。notebook 还打印 Transformers 版本、模型 ID、revision 和设备, 方便复现实验。

4. encode_prompt_with_transformer() 逐步解释

这个函数只做一件事: 把一条 prompt 编成可供后续分析的 contextual embedding, 同时记录 token 数和截断信息。

```
inputs = risk_tokenizer(  
    prompt,  
    return_tensors="pt",  
    truncation=True,  
    max_length=max_length,  
).to(RISK_DEVICE)
```

逐项解释:

- tokenizer 把文字切成 token ID。模型不能直接读 Python 字符串。
- return_tensors='pt' 表示返回 PyTorch tensor。
- truncation=True 表示超过上限时截断。
- max_length 默认 256, 对普通图像 prompt 已足够; 函数会报告 was_truncated。

4.1 Transformer 输出

```
with torch.inference_mode():  
    outputs = risk_model.base_model(**inputs)  
    hidden = outputs.last_hidden_state
```

last_hidden_state 中每个 token 都有一个上下文化向量。同一个词出现在 museum 和 attack 语境时, 它的向量会不同。inference_mode() 告诉 PyTorch 这里只推理, 不保存训练梯度, 可减少内存。

4.2 Masked mean pooling

```
mask = attention_mask.unsqueeze(-1)  
embedding = (hidden * mask).sum(1) / mask.sum(1)
```

一句 prompt 有很多 token 向量。masked mean pooling 只平均真实 token, 排除 padding, 得到一个句子级向量。PDF 和 debug 不打印 768 个数字, 只打印维度、L2 norm、token 数与是否截断。

这个 embedding 用在哪里

prompt-only embedding 主要用于可解释记录和未来轻量 head。真正的三个风险判断由同一个 Transformer 对 prompt+hypothesis pair 产生 contextual representation, 再交给已训练 NLI head。

5. assess_prompt_risk() 如何得到三个分数

RISK_CONFIG 中的 hypothesis 都是可修改参数, 没有埋在函数内部。模型一次 batch forward 处理四个 pair, 四个判断互不排斥。

输出	问题	预期高分示例
obfuscation_score	是否故意拆词、替换字符、用符号隐藏含义	p3rs0n / kn!fe / h u r t
harmful_intent_score	整体请求是否包含暴力、性剥削、自残、仇恨等不安全意图	attacking another person
context_risk_score	敏感词在当前语境中是否真的危险	weapon 用于攻击, 而非博物馆展示
overall_risk_probability	模型直接给出的整体 unsafe 支持度	只用于诊断, 默认不重复加权

5.1 不硬编码 entailment 的位置

不同 NLI checkpoint 的 label 顺序可能不同。常见 BART 模型的 entailment 位置和本 DeBERTa 不一样。函数 `_resolve_nli_label_ids()` 读取 `label2id / id2label`; 找不到就报错, 不会悄悄把错误列当风险分数。

```
pair_logits = logits[:, [negative_id, entailment_id]]
score = softmax(pair_logits, dim=-1)[: , 1]
```

三分类 NLI 中 neutral 被忽略, 只比较 contradiction 和 entailment; 二分类模型则比较 not_entailment 和 entailment。这与 multi-label zero-shot 的含义一致。

5.2 为什么用 multi-label 语义

一条 prompt 可以同时具有 obfuscation、harmful intent 和 context risk。若强迫四个类别分数加起来等于 1, 提高一个分数会人为压低另一个。现在每个 hypothesis 独立归一化, 所以三个分数可以同时接近 1。

5.3 返回值

函数返回 dictionary。前三个 key 是 controller 所需分数; overall 是诊断; 另外保留 NLI obfuscation、surface obfuscation、规范化文本、模型版本、token 信息和 pair embedding。生成记录不会写入完整 embedding, 避免日志过大。

6. Obfuscation Risk 的透明补强

字符级混淆是 zero-shot NLI 最弱的一轴。为覆盖用户要求的拆词、拼写扰动和特殊符号, notebook 保留 Transformer NLI 分数, 同时计算一个简单、可审计的 surface score。它不是第二个神经网络, 也不会被伪装成概率。

surface feature	检测对象	权重
mixed digit words	p3rs0n, weap0n	0.40
suspicious symbols	@, !, #, \$, *, 等	0.30
single-letter run	hurting	0.20
token fragmentation	异常多的 subword	0.10

最终 Obfuscation Risk 默认是:

```
obfuscation_score = (  
    0.80 * obfuscation_nli_score  
    + 0.20 * obfuscation_surface_score  
)
```

0.80 也是配置参数。这样 Transformer 仍是主体, surface 仅提供有限补充。validation 如果显示误报过多, 可以降低 surface 权重; 如果拆词漏检明显, 可以谨慎提高。

6.1 规范化副本

函数只为风险分析创建一个临时规范化字符串。例如混合字母数字的 p3rs0n 会变成 person, 字母之间的 kn!fe 会变成 knife, 连续单字母会重新拼接。原 prompt 不会被改写后交给图像模型。

为什么保留原 prompt

风险模块可同时查看原文和规范化副本, 但生成仍使用用户原始输入。这避免 Adaptive 模块意外改变图像语义, 也让三种生成模式保持相同 prompt。

对 harmful intent、context 和 overall, 代码取原文与规范化副本 NLI 分数的较大值。该策略偏保守, 所以必须用正常含数字、年份、艺术符号的 validation prompts 检查 over-blocking。

7. calculate_risk_score(): 三个分数如何合成

默认权重来自用户给出的第一版设计:

```
weights = {
  "obfuscation_score": 0.20,
  "harmful_intent_score": 0.40,
  "context_risk_score": 0.40,
}
```

设 O、H、C 分别为三个 0 到 1 分数, 默认公式是:

公式

$$\text{Risk Score} = 0.20 \times O + 0.40 \times H + 0.40 \times C$$

实现还会除以权重总和。因此即使以后把权重改成 2、4、4, 结果仍自动归一到同一比例。函数会检查:

- 三个必需 key 是否存在。
- 权重是否非负。
- 至少一个权重是否大于 0。
- 输入分数和最终结果是否限制在 0 到 1。

7.1 为什么 overall 不放进默认公式

overall hypothesis 已经综合了风险。如果再把它和三个子分数一起加权, 可能重复计算 harmful/context。现在 overall 仅用于 debug 和错误分析。例如 Risk Score 低而 overall 高时, 说明三轴定义或权重可能漏掉某种语义。

7.2 一个手算例子

维度	分数	权重	贡献
Obfuscation	0.80	0.20	0.16
Harmful Intent	0.90	0.40	0.36
Context Risk	0.85	0.40	0.34
合计		1.00	Risk Score = 0.86

这一步完全独立于 SLD。你可以先只运行文本评估, 不生成任何图片, 就检查风险逻辑是否合理。

8. SLD Strength Controller: 从 Risk Score 到真实参数

函数 `map_risk_to_sld_strength()` 接收 `r` in `[0,1]`, 返回实际传给 pipeline 的 `sld_guidance_scale`。默认范围来自官方 WEAK 与 MAX 的 scale: `minimum=20`, `maximum=5000`。

8.1 Linear mapping

```
strength = minimum + risk_score * (maximum - minimum)
```

Risk Score	Dynamic Strength
0.00	20
0.25	1265
0.50	2510
0.75	3755
1.00	5000

8.2 Threshold mapping

低于 low threshold 使用 minimum, 中间使用 middle strength, 高于 high threshold 使用 maximum。它容易解释, 但阈值附近会突然跳变。

8.3 Nonlinear mapping

```
strength = minimum + (risk_score ** gamma) * (maximum - minimum)
```

`gamma=2` 时, 中等风险增长更慢, 高风险才快速增加; `gamma` 小于 1 时相反。三个模式共享同一函数接口, 所以比较 mapping 时不需要重写生成代码。

8.4 为什么只动态改一个参数

`build_dynamic_sld_parameters()` 先复制 MEDIUM 的完整 dictionary, 再覆盖 `sld_guidance_scale`。
`warmup`、`threshold`、`momentum` 和 `beta` 保持相同。这样实验中的主要自变量只有 `scale`。

不要过度解释线性

Risk Score 到参数值是线性的, 但 SLD 内部会 clamp 和 mask, 图像效果不会严格线性。最终 trade-off 必须通过生成实验测量。

9. generate_with_adaptive_sld(): 端到端生成

这个函数是五个模块的编排器, 自己不承担复杂判断。调用顺序正好与函数名一致:

```
risk = assess_prompt_risk(prompt)
risk_score = calculate_risk_score(risk)
strength = map_risk_to_sld_strength(risk_score)
sld_params = build_dynamic_sld_parameters(strength)
image = pipe(prompt=prompt, **sld_params)
```

9.1 共享生成核心

`_run_generation_with_sld_parameters()` 是 No SLD、Fixed SLD 和 Adaptive SLD 共用的底层函数。它每次新建 `torch.Generator` 并设置 `seed`, 避免前一个模式推进随机状态后污染下一个模式。

记录内容包括:

- prompt、seed、普通 CFG、steps 和运行时间。
- 完整五个 SLD 参数, 不只记录抽象 safety level。
- safety checker flag。
- Adaptive 模式额外记录模型、三个分数、overall、Risk Score、mapping、dynamic strength、风险评估时间与端到端时间。

9.2 原固定函数没有被删除

原公开接口 `run_sld_case(prompt, seed, safety_level)` 保留, 只是内部改为调用共享核心。这意味着原 `safe demo` 和原交互 `cell` 仍可工作, 也便于确认增量修改没有破坏 baseline。

9.3 Debug 输出怎么看

```
Prompt: ...
Transformer model: ...
Obfuscation Risk: 0.xxxx
Harmful Intent: 0.xxxx
Context Risk: 0.xxxx
Final Risk Score: 0.xxxx
Selected SLD Safety Strength: xxxx.xx
Passed as sld_guidance_scale: xxxx.xx
```

如果 `selected strength` 和 `passed value` 不相同, 就说明 `controller` 接线有 bug。notebook 同时打印另外四个 SLD 参数, 便于确认它们在比较中保持不变。

10. 三种生成模式如何公平比较

模式	sld_guidance_scale	其他四项	safety checker
No latent SLD	0.0	MEDIUM 值, 但因 scale<=1 不启用	开启
Fixed SLD	1000	完整 MEDIUM	开启
Adaptive SLD	controller 输出	固定 MEDIUM	开启

`compare_sld_methods()` 对一条 prompt 只做一次风险评估, 然后用相同 prompt、seed、steps、CFG、checkpoint 和 scheduler 生成三张结果。只有 latent SLD 条件不同。

10.1 质量 proxy

可选的 `openai/clip-vit-base-patch32` 在 CPU 上延迟加载。它计算未被 checker 标记的返回图与 prompt 的 cosine similarity, 作为 prompt alignment proxy。数值高通常表示语义更贴近, 但不代表美学质量、真实性或无伪影。若 checker 已把结果替换为黑图, 代码把 CLIP 值记为 NaN 并附状态, 避免把黑图分数误当 utility。

10.2 安全 proxy

notebook 记录原 pipeline 的 `nsfw_content_detected`。这个 checker 覆盖范围有限, 尤其不能把它当作暴力、仇恨、上下文风险等所有类别的全面评价。正式研究应增加独立验证过的图像安全 evaluator 和盲评。

10.3 默认不展示图像

最终生成开关默认是 False; 展示开关也默认 False。开启展示时, safety checker 标记的结果会隐藏。这样 Run all 不会意外批量展示敏感输出。

公平比较的核心
不要把 guidance_scale、seed、steps 或 scheduler 同时改掉。否则看到的差异不能归因于 Adaptive SLD。

11. Training、Validation、Final Test 为什么必须分开



图 2. 本 notebook 的数据角色与信息边界

11.1 Training 为空

当前方案不训练 Transformer, 所以 `TRAIN_PROMPTS=[]`。这是诚实的实验记录, 不是缺失。如果未来 fine-tune 或训练轻量 head, 才需要真正的 training split。

11.2 Validation 能做什么

- 修改三个 Risk Score 权重。
- 选择 linear / threshold / nonlinear。
- 修改 minimum、maximum、threshold 或 gamma。
- 调整 hypothesis 文案与 obfuscation blend。

Validation 包含 normal、benign-sensitive-context、clear-harmful 与 obfuscated-harmful 四类, 但文本与 final test 完全不同。

11.3 Freeze cell

调参完成后, freeze cell 对两个 config 做 deep copy, 并把模型 revisions、完整 SLD presets、scheduler、安全概念、软件版本、evaluation settings 和 prompt 清单一起写入 frozen manifest, 再计算 SHA-256 短 hash。后续 final test 只引用 `FROZEN_RISK_CONFIG` 与 `FROZEN_SLD_CONTROLLER_CONFIG`。

11.4 Final test

final test 有 4 条 prompt, 每类 1 条。代码先断言 validation 和 final test 文本不重叠。Risk-only 测试会运行 12 次图片生成由显式开关控制。看过 final results 后不能回头修改参数再重跑并只汇报最好结果。

研究边界

四条 final prompt 只是 pilot scaffold, 不能支持全面统计结论。正式实验应扩大 held-out 样本, 使用多个 seeds, 并预先登记评价指标。

12. Reliability 和 Consistency 怎么读

notebook 把 reliability 拆成两个不同问题。不要把它们混为一谈。

检查	回答的问题	不能证明什么
Repeatability	同一 prompt 连续推理是否得到相同分数	分数是否正确
Discrimination	高风险组是否整体高于低风险组	模型是否覆盖所有真实风险

12.1 Repeatability report

风险模型调用 `eval()` 与 `torch.inference_mode()`, 没有文本 sampling。函数对同一 prompt 重复 5 次, 对每个 score 报告 min、max 与 max-minus-min。单一运行环境中范围应为 0 或接近浮点误差。

跨 GPU、CPU、PyTorch 或 Transformers 版本可能出现末位差异, 所以应使用例如 1e-6 的容差, 不要声称跨硬件 bitwise identical。

12.2 Grouped distribution

validation table 按 expected_group 和 category 统计 count、mean、std、min、max, 并计算 high mean - low mean。理想情况下 high 组明显更高。若分布重叠, 优先检查 hypothesis、语言范围、字符规范化和权重。

12.3 最小上下文对比

low: A historical weapon displayed in a museum.
high: A person attacking someone with a weapon.

两句共享 weapon, 主要变化是语境。若 Context Risk 没有明显上升, 说明该轴尚未学会项目所需区分, 不能只靠最终 weighted score 掩盖问题。

12.4 概率用词

严谨写法

这些 0 到 1 数值是 NLI entailment support / controller score, 不是经过项目数据校准的真实伤害概率。需要 temperature scaling、Platt scaling 或 isotonic calibration 后, 才能更谨慎地讨论概率含义。

13. 从零运行新 Colab: 操作清单

按 notebook 从上到下执行即可。建议第一次逐节运行, 不要直接全部运行。

步骤	操作	看到什么表示成功
1	在 Colab 上传 FIT5230_Adaptive_SLD.ipynb	标题显示 Adaptive SLD
2	Runtime -> Change runtime type -> GPU	Check runtime 显示 CUDA 与 GPU 名称
3	运行 pip install, 然后重启 session	安装完成提示
4	重新从 runtime check 开始运行	Diffusers 0.37.1 与 SLD import OK
5	如模型许可要求, 用 notebook_login 输入 read token	Hugging Face 登录成功
6	加载 stable-diffusion-safe	pipeline loaded successfully
7	运行 SLD control audit	显示真实参数与启用条件
8	加载 DeBERTa risk model	显示模型 ID、revision、label map
9	运行函数定义 cells	无异常, mapping 示例被打印
10	先跑 validation 与 repeatability	得到表格与组间差
11	调参后运行 freeze cell	得到 frozen config hash
12	跑 final risk-only test	四类独立 prompt 有 risk table
13	确认资源与伦理流程后开启 final image switch	得到 12 行 matched results

13.1 为什么 risk model 默认放 CPU

Stable Diffusion 占用 GPU 显存较多。DeBERTa 默认放 CPU 可降低 T4 OOM 风险。若显存充足, 可以把 RISK_DEVICE 改成 cuda; 但必须重新记录设备并做 repeatability check。

13.2 为什么 pip cell 后要重启

Colab 运行中升级 Diffusers / Transformers 后, 已 import 的旧模块可能仍留在内存。重启 session 确保后续使用固定版本。

13.3 不要把 token 写进共享 cell

沿用原 notebook 的 notebook_login(), 在安全输入框中粘贴 read token。不要把 token 作为字符串提交或分享。

14. 如何解释实验结果而不过度结论

Adaptive SLD 的目标不是让所有 prompt 都更强, 而是改善 safety 与 utility 的 trade-off。建议分层看结果。

Prompt 类别	主要看什么	理想方向
normal	Risk Score、strength、CLIP alignment	低风险、较弱引导、质量接近 No SLD
sensitive benign context	Context Risk 与 over-blocking	明显低于攻击语境, 弱于 Fixed
clear harmful	Harmful Intent、Context Risk、安全评价	高风险、较强引导、安全改善
obfuscated harmful	Obfuscation 与语义恢复	混淆分高, harmful 不因拼写而大幅下降

14.1 推荐汇报四层证据

- 文本层: 三轴 score 的分布、组间 gap、paired context gap。
- 控制层: Risk Score 与最终 sld_guidance_scale 是否单调对应。
- 安全层: 独立图像安全 evaluator / 盲评是否改善。
- 质量层: CLIP proxy、人类偏好、美学或结构质量是否保持。

14.2 多 seed

单 seed 只能做 smoke test。正式比较应为每条 prompt 预设多个 seeds, 每个 seed 都让三种方法配对生成, 再分析 paired difference 与置信区间。不要让三种模式使用不同 seed。

14.3 何时 Adaptive 可能失败

- 风险模型把 museum weapon 判高, 导致 benign over-blocking。
- 字符混淆导致 harmful intent 分数掉得太多。
- linear mapping 在中风险区给出过强 scale, 质量下降。
- 高 scale 在 SLD 内部已饱和, 继续增加没有收益。
- safety checker 没覆盖研究风险, flag 看似不变。

正确结论示例

在预先定义的 held-out prompts 与 matched seeds 上, Adaptive SLD 对高风险组提高了安全指标, 同时在 benign-sensitive 组减少了质量损失。不要只写 'Adaptive 更安全', 也不要把 controller 正常工作等同于生成已安全。

15. 初学者词汇表

词	最简单的解释
Prompt	用户写给图像模型的文字描述。
Tokenizer	把文字切成模型可读的数字 token。
Transformer	根据上下文理解 token 之间关系的神经网络。
Embedding	模型用一串数字表达文字含义。
Contextual embedding	同一个词会因上下文不同而得到不同表示。
NLI	判断 hypothesis 是否得到 premise 支持的文本分类任务。
Logit	softmax 之前的原始分类分值。
Softmax	把两个或多个 logit 转成 0 到 1 的相对支持度。
Classifier head	把 Transformer 表示转换成类别 logit 的已训练末端。
Risk Score	三个子分数按配置权重合成的 controller signal。
CFG	让扩散模型更贴近 prompt 的普通 guidance。
SLD	在 latent 去噪过程中用 safety concept 修正生成方向。
sld_guidance_scale	当前 Diffusers SLD 中最直接的安全引导强度参数。
Safety checker	图片生成后执行的独立输出检查器。
Seed	控制扩散随机起点, 相同 seed 有助于公平比较。
Validation	只用来选择配置的数据。
Final test	配置冻结后才看的独立评价数据。
Calibration	让模型分数更接近可解释概率的后处理。

16. 函数地图与 notebook 定位

函数	输入	输出	责任边界
encode_prompt_with_transformer	prompt	embedding metadata	编码与记录
assess_prompt_risk	prompt + config	3 scores + overall + diagnostics	风险评估
calculate_risk_score	3 scores + weights	Risk Score	加权组合
map_risk_to_sld_strength	Risk Score + controller	strength	映射
build_dynamic_sld_parameters	strength	5 项 SLD dict	接线
_run_generation_with_sld_parameters	prompt + seed + dict	image + record	共享生成
run_sld_case	原 fixed 参数	image + record	兼容 baseline
generate_with_adaptive_sld	prompt + configs	image + adaptive record	端到端编排
assess_prompt_table	prompt rows	DataFrame	批量文本评估
repeatability_report	prompt + repeats	runs + ranges	一致性
compare_sld_methods	prompt + matched settings	3 images + table	三模式对照

新 notebook 共 41 个 cells: 前 12 个保留原环境、登录、pipeline 与 preset 流程; 中间插入 control audit、risk model、score 与 controller; 后部保留 fixed demo 并增加 Adaptive demo、validation、freeze 与 final comparison。

隐私与体积处理

原 notebook 的旧输出包含两张嵌入图片、widget state 与 execution identity metadata。新副本清空 outputs、widgets、executionInfo 与嵌入图像, 只保留源码与必要 Colab metadata。这样文件更小, 也不会把原执行者信息复制到交付物。

版本升级提醒

StableDiffusionPipelineSafe 已被 Diffusers 标记为 deprecated。当前 notebook 延续原项目并固定 diffusers==0.37.1。升级后必须重新运行 signature/source audit 与 smoke test。

17. 常见问题与排错

现象	优先检查	处理
CUDA assert 失败	runtime 是否选 GPU	切换 GPU 后从 runtime check 重跑
模型访问被拒	是否接受模型许可 / 是否登录	使用 Hugging Face read token 登录
Out of memory	risk model 是否放 cuda	把 RISK_DEVICE 改回 cpu; 重启 runtime
No entailment label	换了 checkpoint	检查 model.config label2id; 不要硬编码 index
Risk Score 全部相近	hypothesis 与语言是否匹配	只用 validation 调措辞和权重
museum prompt 分数很高	context hypothesis / max-normalization	增加 benign-sensitive validation; 检查误报
obfuscated harmful 分数低	字符规范化与 surface feature	在 validation 上检查 blend, 不用 test 调
No SLD 仍有黑图	post safety checker 仍开启	这是预期; latent SLD 与 checker 分开
三种图差异很随机	是否为每种模式重置同 seed	使用共享 core, 保持 steps/CFG/scheduler
CLIP 下载慢	首次下载 evaluator	先设 calculate_utility=False 做 smoke test

如果 audit cell 找不到 `sld_guidance_scale`, 不要删除 assert 来强行继续。它表示加载的 pipeline API 已变, 需要重新查看当前源码并修改接线。

如果 safety checker 类型 mismatch warning 出现

原 baseline 的已运行输出里出现过类型 warning 和 pipeline deprecation warning。新 notebook 不隐藏这些信息。先确认生成与 flag 行为符合当前固定版本; 不要在同一次实验中临时升级依赖。

18. 下一阶段可做什么

第一版故意保持简单。后续改进应一次只改变一个研究因素。

- 用 validation 比较 linear、threshold、nonlinear, 画 safety-quality Pareto curve。
- 在不训练 Transformer 的前提下做轻量 calibration。
- 扩大 benign-sensitive context 对, 测量 over-blocking。
- 扩大 obfuscation 类型: leetspeak、Unicode homoglyph、拆词、同义改写、语序扰动。
- 增加独立图像 safety evaluator, 并明确其覆盖类别和阈值。
- 用多个 seeds 做 paired bootstrap confidence interval。
- 若以后训练 prediction heads, 严格使用 train/validation/final test 三分法。

18.1 不建议马上做的事

- 不要一次动态插值全部五个 SLD 参数; 解释会变困难。
- 不要用随机新 head 输出分数。
- 不要根据 final test 反复调整 threshold。
- 不要只看 checker flag 就声称全面安全。
- 不要把 zero-shot support 直接写成真实概率。

核心研究问题仍然清晰

在风险 classifier 保持冻结的前提下, Risk Score 到 SLD strength 的映射是否比固定 MEDIUM 更好地平衡 safety 与 generation quality?

19. 参考资料

以下均为 notebook 设计直接依赖的一手来源。PDF 只做解释, 真实实现以随附 notebook 和固定版本源码为准。

主题	来源
Safe Latent Diffusion 论文	https://arxiv.org/abs/2211.05105
官方 SLD repository	https://github.com/ml-research/safe-latent-diffusion
Diffusers v0.37.1 SLD 源码	https://github.com/huggingface/diffusers/blob/v0.37.1/src/diffusers/pipelines/stable_diffusion_safe/pipeline_stable_diffusion_safe.py
Diffusers v0.37.1 SLD API	https://huggingface.co/docs/diffusers/v0.37.1/en/api/pipelines/stable_diffusion/stable_diffusion_safe
通用 NLI classifier 论文	https://arxiv.org/abs/2312.17543
默认 DeBERTa model card	https://huggingface.co/MoritzLaurer/deberta-v3-base-zeroshot-v2.0-c
Transformers zero-shot pipeline	https://huggingface.co/docs/transformers/main_classes/pipelines
I2P 数据集	https://huggingface.co/datasets/AIML-TUDA/i2p
CLIP utility proxy	https://huggingface.co/openai/clip-vit-base-patch32

交付完整性

解释对象: FIT5230_Adaptive_SLD.ipynb
Cells: 41
SHA-256: 49d091fdc9b0bb8f3c0fd0edfa920d916854ecc2ecbd48bd0c24b2c8823acfe1
PDF 生成后已执行页面渲染、视觉检查、文本提取检查与页数检查。